

Phonebook Lookup — Group Report (Parts A, B, D)

Group G4 — Searching & Hash tables (§2.2) · Linear · Binary · Interpolation · Hashing & collisions · Demo: phone-book lookup ($n \geq 100k$) · Members: · Build/run & machine spec: README.md

Parts A, B, D of C0B_Group_Project_Guide.pdf. Contribution & AI-use declarations in Appendix.

Part A — Position & purpose

A.1 Syllabus map

Structure	Ref	LO	Built on	Built on top
Linear search	§2.2.2 p.3	LO2	unsorted array	— (baseline; inner loop of hash chains)
Binary search	§2.2.3 p.3	LO2	sorted array	ancestor of BST/B-trees
Interpolation search	§2.2.4 p.4	LO2	sorted array, uniform keys	—
Hashing & collisions	§2.2.5 p.4	LO2+LO3	array buckets + linked-list chaining	dicts, caches, DB hash indexes

Refs: course syllabus 3-Data Structure and Algorithm-CLC.pdf (INT1306_CLC, 2026-09-11); CLRS 4th ed. Ch.11 Hash Tables (§11.1–11.4), MIT Press 2022 (3rd ed. pp.253–280).

A.2 Motivation

Linear search — finds a record with no order/index by scanning until `phone==target` (`phonebook.cpp:250`). Fine for dozens, unusable at 100k (up to 100k compares per lookup). Baseline every other structure improves on; survives as hash-chain scan (`hashtable.cpp:hashSearch`). Wrong when lookups are frequent/large or data is sorted (binary dominates).

Binary Search

- **Mechanism & Efficiency:** Exploits the sorted order of the dataset by halving the search space with each comparison. For a dataset of 100,000 records, binary search requires at most 17 comparisons, compared to 100,000 in linear search.
- **Underlying Structure & Overhead:** Implemented on a sorted contiguous array. The initial sorted index requires a one-time build cost of $O(n \log n)$ via `buildSortedIndex()`. Subsequent insertions incur an $O(n)$ overhead due to `std::vector` element shifting (`phonebook.cpp:177`), despite finding the position in $O(\log n)$ via `lowerBound()`.
- **Architectural Role:** Serves as the conceptual foundation for advanced tree-based indexing structures (e.g., BST, B-Trees).
- **Limitations:**
 - Inefficient for unsorted data when one-off lookup costs are lower than sorting costs.
 - Poorly suited for write-heavy workloads with frequent mutations due to the $O(n)$ insertion overhead (where Hash Tables achieve $O(1)$).
 - Suboptimal for pure exact-match lookups where order is irrelevant.
- **Interpolation Search**
 - **Mechanism & Expected Complexity:** Probes positions based on the numerical value of the target relative to the range bounds, achieving an expected time complexity of $O(\log \log n)$ on uniformly distributed keys (outperforming Binary Search's $O(\log n)$). It addresses Binary Search's limitation of dividing the search space strictly in half regardless of the target's value.
 - **Limitations & Degeneracy:**
 - **Data Distribution Sensitivity:** On non-uniform or highly skewed distributions, the search space reduction degrades to $O(n)$, probing elements one by one.
 - **Computational Overhead:** On small datasets (n), the constant factor required to compute the interpolation formula outweighs the simpler midpoint operations of Binary Search.
- **Project Scope Note:** Because real-world phone number strings do not follow a uniform distribution, Interpolation Search is evaluated strictly from a theoretical standpoint (see §2.2.4) and is omitted from the implementation in `src/`.

Hash Table & Collision Resolution

- **Performance & Scalability:** Provides expected $O(1)$ time complexity for lookup, insertion, and deletion regardless of dataset size n . Unlike linear search ($O(n)$ lookups) or binary search (which incurs an $O(n)$ insertion penalty to maintain array order), a hash table efficiently handles high-write workloads where records are added continuously.
- **Implementation Details (`hashtable.cpp`):**
 - **Collision Resolution:** Implements separate chaining using a dynamically resized bucket array, where each bucket points to a singly linked list. Collisions are resolved by linear traversal within the individual chain.
 - **Hash Function:** Utilizes a 64-bit polynomial rolling hash (`hash * 31 + (c - '0')` with standard unsigned integer overflow wrapping) mapped via modulo arithmetic: `% numBuckets (HashForSize)`.
 - **Dynamic Resizing:** Initializes at size 101. When the load factor exceeds 0.75, the table rehashes to the next prime number approximately double its capacity (`nextPrime(2 * currentSize)`), utilizing a $6k \pm 1$ primality test (`isPrime`).
- **Architectural Trade-offs & Limitations:**
 - **Prefix & Range Queries:** Incapable of ordered traversals or prefix searches (e.g., retrieving phone numbers starting with "09"); such operations require a sorted array, Trie, or balanced BST.
 - **Worst-Case Degradation:** Pathological collision patterns can degrade lookup performance from $O(1)$ to $O(n)$, whereas self-balancing search trees strictly guarantee $O(\log n)$.
 - **Memory Overhead:** Maintaining a low load factor to minimize collisions results in unused bucket allocations and pointer overhead from linked list nodes.

Part B — Algorithms & complexity

B.1 Operation table

Demo implements linear, binary (sorted phone index), and hash table; interpolation is topic (§2.2.4) included for completeness but not implemented (needs uniform keys). A number without justification scores nothing.

Operation	Best	Avg	Worst	Space	Why
Linear search by phone	$O(1)$	$O(n)$	$O(n)$	$O(1)$	No index — scan until found; first hit $O(1)$, miss scans n .
Binary search by phone	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Halves range per probe; needs sorted array.
Interpolation search	$O(1)$	$O(\log \log n)$	$O(n)$	$O(1)$	Value-proportion probe; $O(\log \log n)$ only if uniform, else $O(n)$.
Hash insert	$O(1)$	$O(1)$	$O(n)$	$O(1)$	Hash → bucket, append to chain; $O(n)$ if all collide.
Hash search	$O(1)$	$O(1)$	$O(n)$	$O(1)$	Hash → bucket, walk chain; expected $O(1)$, worst single chain.
Hash delete	$O(1)$	$O(1)$	$O(n)$	$O(1)$	Find in chain then unlink; same as search.
Rehash (load>0.75)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Reinsert all keys into doubled buckets; amortised $O(1)/\text{insert}$.

B.2 Implementation notes (C++ source)

- **CSV** `phonebook.cpp:30-83`: quoted "Do, Thanh Tuan" with `"` escapes, unquoted split on last comma, trim/strip quotes/`\r`, skip blank/empty-name/non-digit-phone.
- **Insert** `143-164`: reject empty/non-digit/duplicate phone (hash check), store `CapitalizeFirst(ToLower(name))` + hash index.
- **Delete** (opt 8): $O(n)$ vector erase + full hash rebuild; missing phone → `Phone number not found`.
- **Search**: `searchLinearByPhone` exact scan; `searchHashByPhone` chain lookup; `searchBinaryByPhone` binary search on sorted index ($O(\log n)$ → hash resolve); `searchLinearByName` case-insensitive scan.

- **Sorted index** `buildSortedIndex`: bulk load then merge-sort once $O(n \log n)$; per-insert `lowerBound` $O(\log n)$ +shift $O(n)$ with split hash / sorted-index timing; delete rebuilds.
- **Hash** `hashtable.cpp`: poly hash 64-bit wrap % `numBuckets`; $101 \rightarrow \text{nextPrime}$; rehash $0.75 \rightarrow \text{nextPrime}(2\times)$; `isPrime` $6k \pm 1$.
- **Timer** `timer.hpp`: `timeIt ms, benchmark(work,5)` best-of-5, `printTaskDuration` $\rightarrow$ `\nTook: Xms.`
- **Benchmark** `main.cpp:11-63` opt 0: auto-loads CSV, picks `first/middle/last/miss` phones (`miss=0000000000` true worst for hash/binary), times linear/hash/binary $\times 5$.

Part D — Cross-language comparison

D.1 Scope & file map

Same chained hash table + linear scan + binary on sorted index over `contacts_100k.csv` ($n=100k$) in all five languages — no `dict/map/HashMap`. This is a *language+runtime* comparison of equivalent code, not C++ vs Python's built-in `dict` (different question).

C++	Python	Go	JS (Node)	Java
<code>src/main.cpp</code>	<code>python/phonebook/main.py</code>	<code>go/phonebook/main.go</code>	<code>javascript/phonebook/src/main.js</code>	<code>java/phonebook/src/com/phonebook/Main.java</code>
<code>src/phonebook.cpp + phonebook.hpp</code>	<code>phonebook.py</code>	<code>phonebook.go</code>	<code>src/phonebook.js</code>	<code>PhoneBook.java</code>
<code>src/hashtable.cpp + hashtable.hpp</code>	<code>hashtable.py</code>	<code>hashtable.go</code>	<code>src/hashtable.js</code>	<code>HashTable.java</code>
<code>include/timer.hpp</code>	<code>timer.py</code>	<code>timer.go</code>	<code>src/timer.js</code>	<code>Timer.java</code>
<code>include/contact.hpp</code>	<code>contact.py</code>	<code>contact.go</code>	<code>src/contact.js</code>	<code>Contact.java</code>

Differences are only runtime-forced (64-bit wrap, stdin, timing API).

D.2 Comparison table

Criterion	C++	Python	Java	JavaScript	Go
Built-in structures	<code>vector</code> (contacts, buckets, sorted index)	<code>list</code> (contacts, buckets, sorted index)	<code>ArrayList</code> (contacts, sorted index) + <code>HashNode[]</code> (buckets)	<code>Array</code> (contacts, buckets, sorted index)	<code>slices []Contact, []*HashNode, []string</code>
Effort	highest: <code>new/delete</code> , header split, <code>=delete</code> , $6k \pm 1$, rehash, CSV state machine, <code>lowerBound</code> , <code>CMake -O2</code>	lowest: <code>dataclass+list</code> , & <code>MASK64</code> , <code>""</code> -CSV, <code>lowerBound</code> , <code>perf_counter</code>	high: <code>remainderUnsigned</code> , <code>HashNode[]</code> , NIO.2 CSV, <code>lowerBound</code> , <code>nanoTime</code> , <code>javac -d</code>	high: <code>BigInt</code> & <code>MASK64</code> , <code>perf_hooks</code> , <code>readline CLI</code> , <code>mulberry32(42)</code>	medium: <code>uint64</code> wraps free; chaining/CSV/ <code>lowerBound/time.Now</code> , <code>go run</code>
Runtime	D.3/D.7	D.3/D.7	D.3/D.7	D.3/D.7	D.3/D.7
Memory	lowest: ~165 MB heap at 1M (Massif)	~2× C++: ~340 MB at 1M (tracemalloc)	~2× C++: ~350 MB HeapNuse at 1M (pprof)	live ~260 MB at 1M, RSS baseline ~56 MB	polled ~242 MB at 1M; RSS ~323 MB
Mgmt	manual RAII, deterministic, no GC	refcount + gen. GC	tracing G1, concurrent	V8 gen. GC	G1, may pause
Types	strong static <code>constexpr/const</code> , <code>template</code> , <code>-Wall</code>	dynamic + hints (<code>dataclass</code> , <code>List/Callable</code>)	static+generics, checked <code>IOException</code>	dynamic JS <code>'use strict'</code> , no TS	static concrete, no generics, errors as <code>-1/nil</code>
Readability	verbose/explicit; costs visible	most concise; with <code>open</code> , f-strings	verbose enterprise; ~2× Python LOC	small, C++-mirroring	explicit; <code>defer/copy</code> -shift noisy but clear
Use when	perf-critical / 1M / embedded: <code>-O2</code> , no GC	teaching/prototyping/glu e; not hot loop (use <code>dict</code>)	large-team/enterprise/Android; not terse scripts	demo/web/zero-build CLI; not 100k+ hot path (<code>BigInt</code>)	backend/microservice: single binary, <code>pprof</code> , GC

D.3 Runtime growth

Best-of-5 ms at `last` (index $n-1$, same phone every lang) across $n=50/10k/100k/200k/500k/1M$ (log-log). Linear $\sim$ linear with n ; hash flat $O(1)$, binary near-flat $O(\log n)$, both position-independent unlike linear.

 Runtime vs n (5 langs, log-log)

Per-language detail (every algo $\times$ every case vs n , and best/avg/worst per algo) is generated by `benchmark/plot.py` — see `benchmark/gallery.html` and `benchmark/plot-unified-*.png` / `plot-per-algo-*.png` (10 images, omitted here for length; all 28 PNGs remain in `benchmark/`).

D.4 Memory profiling

Headless on same load path (CSV $\rightarrow$ contacts+hash+sorted index) at every n , plus kernel peak RSS (`wait4/ru_maxrss`) as uniform yardstick. Reproduce: `make run-memory`; raw evidence (massif+ms_print, tracemalloc, pprof, `.heapprofile`, JFR) in `benchmark/mem/`.

 Peak memory (profiler heap, top; RSS, bottom)

n	C++ Massif	Python tracemalloc	Go pprof HeapInuse	JS V8 heapUsed	Java polled heap
50	0.1	0.2	0.8	4.8	9.5
10k	2.2	3.4	4.1	7.3	11.6
100k	18.5	34.2	37.3	49.0	29.4
200k	36.9	68.5	73.4	64.4	44.9
500k	82.5	169.7	177.7	135.9	143.7
1M	164.9	339.7	350.3	260.2	241.9

Peak RSS (MB, same batch):

n	C++	Python	Go	JS	Java
50	12.6	16.6	12.7	55.8	53.5
10k	12.9	20.4	12.9	74.8	61.6
100k	21.3	53.7	25.4	157.0	85.6
200k	39.3	91.1	46.5	191.1	128.0
500k	88.5	202.7	101.7	282.8	214.4
1M	174.9	386.6	239.8	441.9	323.4

Why: **C++** floor — contiguous `vector<Contact>` + SSO, $\sim$ 165 MB at 1M is the data. **Python** $\sim$ 2 $\times$ — every object boxed `PyObject` + header. **Go** $\sim$ 350 MB HeapInuse — `[]Contact` values but per-`HashNode` allocs; RSS lower due to span accounting. **JS** largest baseline $\sim$ 56 MB RSS at $n=50$; live $\sim$ 260 MB but RSS $\sim$ 442 MB (V8 heap reservation). **Java** $\sim$ 54 MB baseline (JVM+G1), $\sim$ 242 MB polled heap at 1M. Honest claim: data cost ranks $C++ < Python \sim Go < JS < Java$; whole-process cost dominated by baselines at small n . Both panels needed.

D.5 Statistical fairness

Every cell in `benchmark/results.csv` holds 5 raw runs (`timeIt`, no discard) — 360 cells $\times$ 5=1800 rows (5 langs $\times$ 6 sizes $\times$ 4 cases $\times$ 3 algos). Opt 0 reports best-of-5; CSV keeps all 5 for mean/stdev/CV offline. No discarded warm-up yet, so run 1 includes cold start. Reproduce: `python3 -c with csv.DictReader, statistics.mean/stdev, CV=stdev/mean`.

Representative at $n=1M$ (`last` = linear worst, same phone every lang), ms mean $\pm$ stdev (best):

Lang	Linear-last	Hash-last	Binary-last
C++	4.302890 $\pm$ 0.679485 (3.814707)	0.000212 $\pm$ 0.000392 (0.000035)	0.001013 $\pm$ 0.001940 (0.000125)
Python	42.768813 $\pm$ 0.532619 (41.975203)	0.003053 $\pm$ 0.003243 (0.001388)	0.004541 $\pm$ 0.003323 (0.002941)
Go	3.468332 $\pm$ 0.126620 (3.300311)	0.000275 $\pm$ 0.000513 (0.000032)	0.000810 $\pm$ 0.001469 (0.000133)
JS	17.269537 $\pm$ 2.736207 (13.113779)	0.003668 $\pm$ 0.004672 (0.001240)	0.005866 $\pm$ 0.006290 (0.002539)
Java	22.521668 $\pm$ 0.718975 (21.738484)	0.001006 $\pm$ 0.001003 (0.000509)	0.004678 $\pm$ 0.004580 (0.002551)

Cold-start first $n=1M$ run1/best: JS 58.5 $\times$ /46.6 $\times$ /25.9 $\times$, Java 11.8 $\times$ /79.1 $\times$ /5.3 $\times$, Python 11.1 $\times$ /5.6 $\times$ /3.6 $\times$, C++ 11.2 $\times$ /22.4 $\times$ /20.5 $\times$, Go 17.3 $\times$ /18.1 $\times$ /19.4 $\times$. Full 60-cell table generated from CSV (not hand-copied).

Why Java slow on run 1: JVM interpreted + class-load/G1, then C1/C2 compile hot loop after ~10k iters — one 100k scan triggers it (middle-linear 39 → 17 ms runs 1–5; tiny hash ops 66× after compile). V8 same (Ignition → TurboFan, settled by run 3–5; first coldest as it runs first). C++/Go AOT (-O2/go build) run-1 excess is cache/branch + sub-μs timer floor (CV 1.4–1.8 on hash/binary). Python no JIT; large linear most stable (CV 0.02–0.04 at 1M).

Fairness: best-of-5 mitigates JIT but ≠ discarded warm-up. Fix: 3 untimed discarded searches (same phone, sink assignment) before timed `r=1..5` in `java/.../Main.java:~150` and `javascript/.../main.js:~76`; keep CSV header/seed 42/miss=0000000000. Until then compare `min(best)` or `mean(runs 2–5)` for Java/JS and state which.

D.6 Measurement method

Opt 0 covers 5 targets and prints best-of-5 per target:

```
[role index N phone P]
  Linear best of 5: Xms. (index N)
  Hash best of 5: Yms. (index N, position-independent)
  Binary best of 5: Xms. (index N, sorted index)
```

miss=0000000000 (not in dataset) — worst for hash (full chain) and binary (log n); middle=n/2 — linear avg, binary best (1 probe).

```
make run-benchmark          # all 5 langs → benchmark/results.csv (overwrite) +
benchmark/plot.png
make run-benchmark-sizes    # ×6 sizes → plot-runtime-vs-n.png + per-lang PNGs
make run-memory             # profilers + RSS → benchmark/mem/results.csv + plot-memory.png
```

Batch mode (60 rows/lang: 4 cases×3 algos×5 runs, target_index, phone auditable; miss index -1):

```
make # builds ./build/cpp/demo
./build/cpp/demo --benchmark-csv benchmark/results.csv data/contacts_100k.csv
python3 python/phonebook/main.py --benchmark-csv benchmark/results.csv --append
data/contacts_100k.csv
(cd go/phonebook && go run . --benchmark-csv ../../benchmark/results.csv --append
../../data/contacts_100k.csv)
(cd javascript/phonebook && node src/main.js --benchmark-csv ../../benchmark/results.csv --
append ../../data/contacts_100k.csv)
(cd java/phonebook && javac -d out src/com/phonebook/*.java && java -cp out com.phonebook.Main
--benchmark-csv ../../benchmark/results.csv --append ../../data/contacts_100k.csv)
```

CSV: language, dataset, n, case, algo, run, ms, timestamp, toolchain, target_index, phone — 300 rows single-size, 1800 multi-size. plot.py parses all ms formats defensively (C++ setprecision(17), Python repr, Go %g, JS, Java) and draws plot.png (3 linear-scale panels, split axes) + line/unified/per-algo PNGs; without matplotlib it prints plot skipped and CSV is still produced.

 Phone search: linear vs hash vs binary (5 langs, best-of-5)

Additional per-case PNGs (plot-first-* ... plot-miss-*, 12 images) remain in benchmark/ and benchmark/gallery.html; omitted here for length.

D.7 Why the gap

- **C++ -O2**: contiguous structs + SSO → tight machine loop, no per-element overhead (baseline).
- **Python**: every Contact/str boxed PyObject + bytecode dispatch + dict lookups → last-linear ~10× (100k interpreter steps vs instructions); hash stays small (O(1) → tax once).
- **Go**: compiled value slices → real machine loop, gap is bounds checks + GC barriers, not interpretation — closest to C++.
- **JS (V8)**: heap-object array + hidden-class checks + JIT warmup (best-of-5 lets TurboFan settle; spread is tier-up).

- **Java:** `ArrayList<Contact>` refs + C2 after ~10k iters (one 100k scan trips it) + G1 pauses; startup not measured (fair — all langs same).

`first/middle/last` hit identical phone every lang (directly comparable). Portable claim: *trend within each lang* — linear grows first→last, hash/binary flat — holds in all five (DSA point). Cross-lang magnitudes are runtime properties; cite only with mechanism + machine spec.

D.8 Recommendation

For a production phonebook (frequent exact-match, 100k–1M, latency+footprint dominate) we pick **C++**: `last-linear` 3.81 ms vs Go 3.30, JS 13.11, Java 21.74, Python 41.98 (D.7/D.10, i5-1135G7, `-O2`); hash/binary ~μs position-independent; memory floor 164.9 MB heap/174.9 MB RSS at 1M vs Go 350.3/239.8, Python 339.7/386.6, JS 260.2/441.9, Java 241.9/323.4 (D.4); no baseline (12.6 MB RSS at n=50 vs JS 55.8/Java 53.5), deterministic RAII, no GC pauses, `-Wall`/Massif control. Trade-off: Go/Java/Python/JS offer faster prototyping/concurrency but pay GC/JIT/baseline; C++ pays manual `new/delete` and debug cost. Industry precedent (MySQL, PostgreSQL in C/C++) supports this for lookup/index engines; if constraint were rapid prototyping/managed concurrency, pick Go.

D.9 Measurement tables

Best-of-5 ms per target (`first/middle/last`) — fresh 100k run on this machine (uncommitted ports; `make run-benchmark` to refresh `results.csv`).

Lang	n	Linear (first/middle/last)	Hash (first/middle/last)	Binary (first/middle/last)
C++	100k	0.000017 / 0.107453 / 0.216328	0.000034 / 0.000034 / 0.000032	0.000102 / 0.000093 / 0.000103
Python	100k	0.000297 / 1.892173 / 3.873464	0.001344 / 0.001210 / 0.001342	0.002711 / 0.002295 / 0.002762
Go	100k	0.000021 / 0.108283 / 0.224579	0.000038 / 0.000038 / 0.000034	0.000145 / 0.000121 / 0.000122
JS	100k	0.000289 / 0.331141 / 1.156205	0.000793 / 0.000993 / 0.000742	0.001739 / 0.001320 / 0.001489
Java	100k	0.000660 / 0.854805 / 1.298942	0.000450 / 0.000482 / 0.000418	0.002910 / 0.001614 / 0.001879

Dataset `contacts_100k.csv`, seed 42, toolchains in D.10.

D.10 Test conditions

Component	Value
CPU	Intel i5-1135G7 @2.40 GHz (4C/8T)
RAM	15 GiB
OS	Omarchy (Arch), kernel 7.2.3-arch1-3
C++	g++ 16.2.1, <code>-O2</code> (Makefile)
Python	3.14.7
Go	1.27.1 linux/amd64
Node	v26.7.0
Java	javac 27 (JDK 27)
Dataset	<code>data/contacts_100k.csv</code> (n=100k), seed 42
Commit	<code>adc54407393a2d8b82260a8af5695ad077df7641</code>

Platform coverage: the benchmark harness was **tested on Linux only** — Windows/macOS are not yet tested. The C++ program is verified on Windows via `cmake --preset mingw-release` (MSYS2 MinGW, manual build flags and CMake preset both checked); the Python/Go/JS/Java ports are expected to work on any OS with the runtime installed, but their numbers here are Linux-only. Do not treat results from an untested platform as comparable.

Appendix

A.1 Test conditions — see D.10. **A.2 Dataset** — Name, Phone per line, no header; `python3 data/generate.py --seed 42`; sizes 50/10k/100k/200k/500k/1M nested prefixes (seed 42); `contacts_50.csv` for demos; see `data/README.md`. **A.3 Ports** — Python `perf_counter/randint/& MASK64/(ord(c)-48)&0xFF`; Go `uint64/time.Now/math/rand/bufio.Reader`; JS `perf_hooks/Math.random/readline/promises/BigInt & MASK64/fs.readFileSync`; Java `nanoTime/1e6/Random/Scanner/Long.remainderUnsigned, out/` `git-ignored`. **A.4 Sources** — CLRS 4th ed. Ch.11 (§11.1–11.4), 3rd ed. pp.253–280, accessed 2026-09-12.

A.5 Contribution

Deliverable	Name	Student Code
C++ core	Nguyễn Văn Trãi Trương Quang Tuấn Nguyễn Phương Quốc Vương	N25DECE085 N25DECE074 N25DECE086
Linear Search	Trần Quốc Anh Nguyễn Quốc Toàn	N25DECE077 N25DECE070
Binary Search	Trần Quốc Anh Nguyễn Quốc Toàn	N25DECE077 N25DECE070
Interpolation Search	Nguyễn Quốc Toàn	N25DECE070
Hash & Hash Function	Nguyễn Phương Quốc Vương Nguyễn Phạm Thành Trung	N25DECE086 N25DECE072
Collision Handling	Nguyễn Văn Trãi Trương Quang Tuấn	N25DECE085 N25DECE074
Language ports (Go / Python / Java / JS)	Nguyễn Văn Trãi Trương Quang Tuấn	N25DECE085 N25DECE074
Benchmark & plotting	Nguyễn Văn Trãi	N25DECE085
Dataset generation	Nguyễn Văn Trãi	N25DECE085
Report & PDF export	Nguyễn Văn Trãi Trương Quang Tuấn	N25DECE085 N25DECE074
Slides	Nguyễn Quốc Toàn Trần Quốc Anh	N25DECE070 N25DECE077
Build system	Nguyễn Văn Trãi Trương Quang Tuấn	N25DECE085 N25DECE074
Exercise	Nguyễn Phương Quốc Vương Nguyễn Phạm Thành Trung	N25DECE086 N25DECE072
Grade the assignment	Nguyễn Phương Quốc Vương Nguyễn Phạm Thành Trung	N25DECE086 N25DECE072

A.6 AI-use — OpenCode/Copilot/web chat for brainstorming + boilerplate translation/tooling. **C++ core** — no AI in `src//include/` (`contact.hpp`, `hashtable.hpp/.cpp`, `phonebook.hpp/.cpp`, `timer.hpp`, `main.cpp`); all AI output reviewed/compiled/executed. AI-assisted: ports (`python|go|javascript|java/phonebook/*`) faithful translations + tooling (`benchmark/plot.py`, `mem_profile.py`, `report/export_pdf.py`). Not AI-assisted: C++ core, report prose/analysis (A/B/D), measurements (`results.csv`, `mem/*`, D.3–D.5/D.9–D.10 — real `make run-benchmark` runs on D.10 machine).